

04A. QA: Best Practices & Standards Manual

 Page is about?

Purpose:** To serve as the universal QA engineering standards playbook for automation teams at GS, — covering **coding standards, framework setup, feature design, locator strategy, SQL validation, test data handling, CSV usage, logging, RCA, and a ready-to-use cucumber + selenium test case sample.

Whether you're writing a brand-new automation flow or maintaining legacy tests across Astro, Unite, Jett, or Universal Enrollment — this doc ensures **consistency, traceability, and high-quality automation** delivery.

Section 1: Automation Coding Standards

Based on: [Automation Standard Practices](#) and [front-end-standards](#) documents

General Java Practices

- **Readable Code Wins:** Write minimal, logical, and modular blocks.
- **Consistent Naming:** Use meaningful variable/method names that follow camelCase.
- **Consistent formatting:** Use IDE tools (Ctrl+Shift+F or Ctrl+I) to auto-indent. *(Always maintain 2–4 space indentation.)*
- **Modular Design:** One method = one job.
- **Avoid Deep Nesting:** Refactor logic into smaller, testable methods & try to use GitHub Copilot for refactoring code & advance java implementation with best practices.
- **Function Size:** A function should ideally perform a single operation.
- **camelCase** for methods and variables; UPPERCASE_SNAKE_CASE for constants.
- **Use helper classes** for shared logic (e.g. LoginHelper, CSVLoader).
- **DRY (Don't Repeat Yourself) principle:** Reuse existing code before writing new methods. *(Refactor repetitive patterns into reusable functions.)*
- **Avoid magic values:** Externalize all SQL, errors, and dynamic input.
- **SQL Queries:** Use UPPERCASE for SQL keywords; lowercase for fields.

Exception Handling

- Use try-catch blocks for automation failures (timeouts, missing elements).
- Always include fallback mechanisms or recovery steps where needed.
- Capture logs/screenshots on failure and report immediately.

Documentation & Comments

- Use inline comments **only where logic is non-trivial**.
- Maintain up-to-date documentation in GitLab or Confluence.
- Include @author, @date, and @description headers.

Javadoc for Every Method

Every method must have a `/** ... */` block:

Javadoc for Every Method Example

```
/**
 * Login user using credentials from test case file.
 * @param testCaseName CSV test case name
 * @prime.usage When user logs in for <test case name>
 */
public void loginWithTestCase(String testCaseName) { ... }
```

*All step definition Javadoc blocks **must** include @prime.usage.*

Section 2: Folder, File, Locator & Plan Structure

Folder Placement

Type	Location
------	----------

Feature File	/features/<plan>/module/
Step Def	/stepdefs/<plan>/
Locators	/element_locators/<plan>/
CSV/TestData	/resources/testdata/<plan>/
SQL Queries	/resources/sql/<plan>/

Use common plan folder when logic applies to all plans.

Element Locator Standards

1. Prefer id css xpath (only if dynamic text).
2. Use replacement tokens: \$\$param_1\$\$, \$\$param_2\$\$.
3. Always avoid:
 - Plan-specific IDs
 - Hard-coded XPath
 - Label clicks (use actual element instead)

Example:

Element Locator Example

css = "td[title*='\$\$param_1\$\$']"
xpath = "//td[contains(text(), '\$\$param_1\$\$')]/../td/button"

Section 3: Feature File Writing Standards (Cucumber)

Feature Header

Feature Header Example

Feature: Oregon: Employer completes bank enrollment

Background:
 Given starting registration URL for employer
 And using browser as desktop
 And set test case creator as swapnil.patil@ascensus.com

Scenario Outline Format

Scenario Outline Format Example

Scenario Outline: Employer Registration with valid data
 When user completes the employer registration flow for <test case name>
 Then verify confirmation number is displayed
 And validate registration SQL for <test case name>

Every **Scenario name must be unique**, or qTest will fail mapping
Use <test case name> from CSV rows
Avoid chaining one scenario's output into another

Section 4: SQL Validation & Error Handling

Database Validation Guidelines

- Validate using \$\$CONFIRMATION_NUM\$\$ when applicable.

- Always include post-action SQL to confirm backend changes.
- Prefer reusable SQL scripts under /resources/sql/common/
- Format SQL with readable aliases and joins.

Externalized Error Messages

Store in /errors/<plan>/, reusable via property references.

- Avoid hard-coding error strings in step definitions.
- Use property key lookups in both Java and Gherkin validations.

Section 5: CSV Strategy, Data Gen & Test Case Logging

CSV File Usage

- File name must match Feature: Registration.csv, BankUpdate.csv.
- Smaller files preferred; split if needed (e.g., Bank.csv, BankValidation.csv)
- Use data generation utility:
 - {RANDOM_SSN}, {RANDOM_EMAIL}, {RANDOM_PHONE}

Logging Used Accounts

Env	Table
Unite	QA_UNITE_TEST_CASES
Astro	QA_SSRP_TEST_CASES (per schema)

SQL Logging Example:

```
INSERT INTO QA_UNITE_TEST_CASES (SEQ_TEST_CASE_ID, SEQ_ACCT_ID, TEST_CASE_PATH, TEST_CASE_NAME,
TEST_CASE_CREATOR, TRAUNCH_ID)
VALUES (SEQ_TEST_CASE_ID_PK.nextval, '1', 'member.registration', 'Member completes Registration', 'Swapnil
Patil', '100001');
```

Section 6: Sample Feature + StepDef: IDP Login Flow

Feature:

Feature: IDP Login Example

Feature: IDP: User Login With Valid Credentials

Background:

```
Given starting login URL for member
And using browser
And set test case creator as swapnil.patil@ascensus.com
```

Scenario Outline: Valid Login Flow

```
When user logs in with credentials for <test case name>
Then verify login successful for <test case name>
```

Step Definition:

Step Definition: IDP Login Example

```
@When("^user logs in with credentials for \"([^\"]*)\"$")
public void loginWithCredentials(String testCaseName) {
    Map<String, String> testData = getTestCaseInformation(testCaseName);
    LoginHelper.loginWithUsernamePassword(this, testData.get("USERNAME"), testData.get("PASSWORD"), false);
}

@Then("^verify login successful for \"([^\"]*)\"$")
public void verifyLogin(String testCaseName) {
    assertTrue(isTextDisplayed("Welcome"));
}
```

LoginHelper.java

LoginHelper.java Example

```
public static void loginWithUsernamePassword(BaseClass scenario, String username, String password, boolean
isCSR) {
    scenario.fillElement("Login User Name Textbox", username);
    if (!isCSR) scenario.clickElement("Continue Button");
    scenario.fillElement("Login Password Textbox", password);
    scenario.clickElement("Login Button");
}
```

Section 7: Logging, Failures & RCA

- All failures must:
 - Log to console and TestNG
 - Save screenshot and attach to Jira ticket
- RCA must include:
 - What failed?
 - Why it failed?
 - What fixed it?
 - Is it preventable?

Maintain weekly RCA doc as Excel or Confluence table.

Section 8: Regression Tagging & Suite Structure

- Use cucumber tags:
 - @smoke, @sanity, @regression, @idp, @csr
- Define groups in TestNG XML:

TestNG XML Example

```
<suite name="IDP Suite">
  <test name="Login">
    <classes>
      <class name="com.qa.idp.LoginStepDefs" />
    </classes>
  </test>
</suite>
```

Final Checklist Before Committing

All Java files formatted and saved
Javadoc on all public methods
No hardcoded data
Locators reusable and in correct folder
All test data and SQL externalized
All feature files validated with cucumber
Scenario names are unique
Screenshots/logs captured on failure
Feature committed + test ran via mvn clean install

Section 9: QA Ownership & Reporting

Shadow-to-Ownership Model

Phase	Task
Week 1	Pair testing, shadow reviews
Week 2–3	Lead one regression/smoke cycle
Week 4	Own defect triage, reporting, KT

Daily QA Rituals

- Attend stand-ups with updates
- Update Jira/qTest by EOD
- Log all test runs, failures, and recovery actions